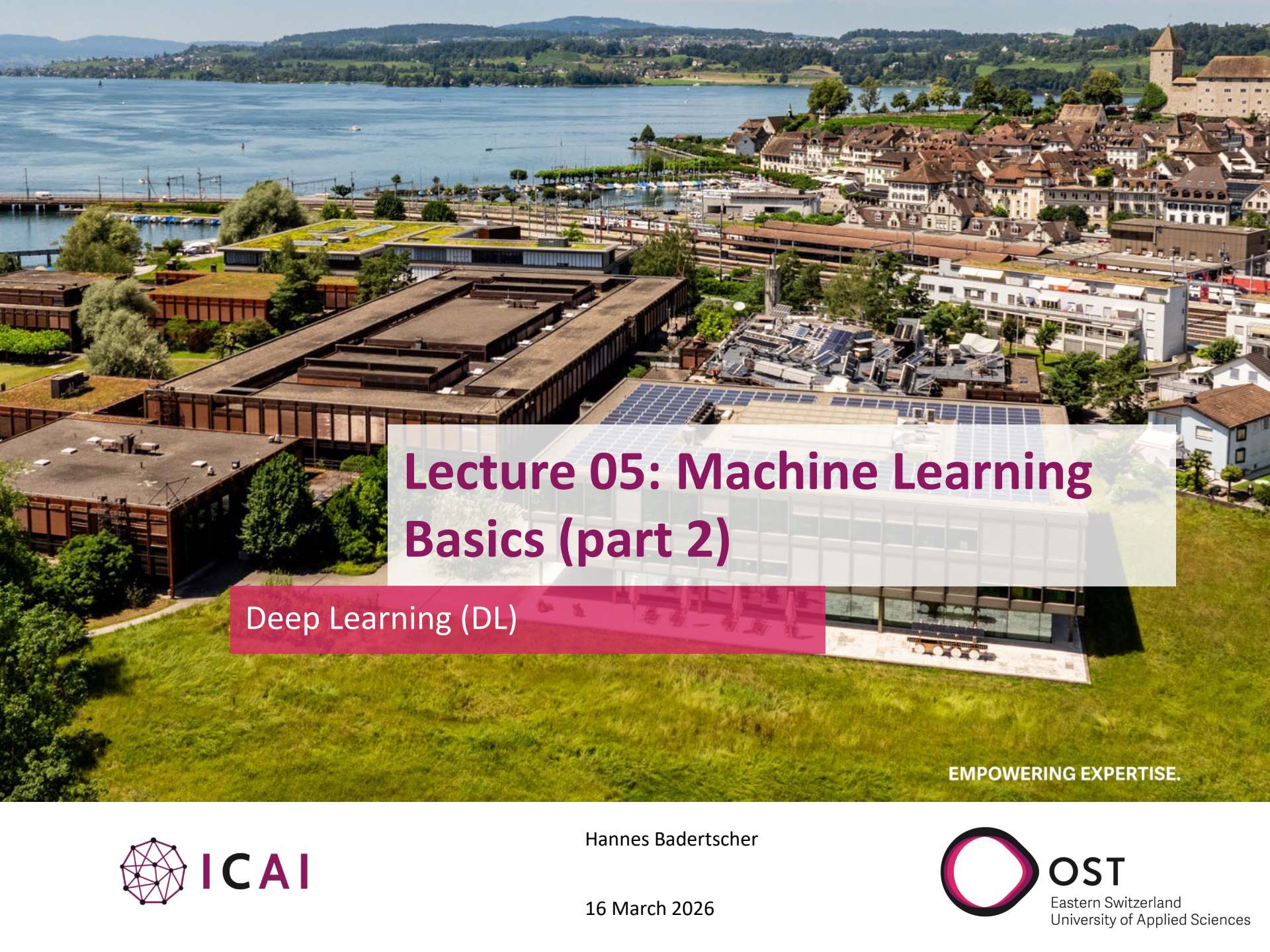




Lecture 05: Machine Learning Basics (part 2)

Deep Learning (DL)

EMPOWERING EXPERTISE.

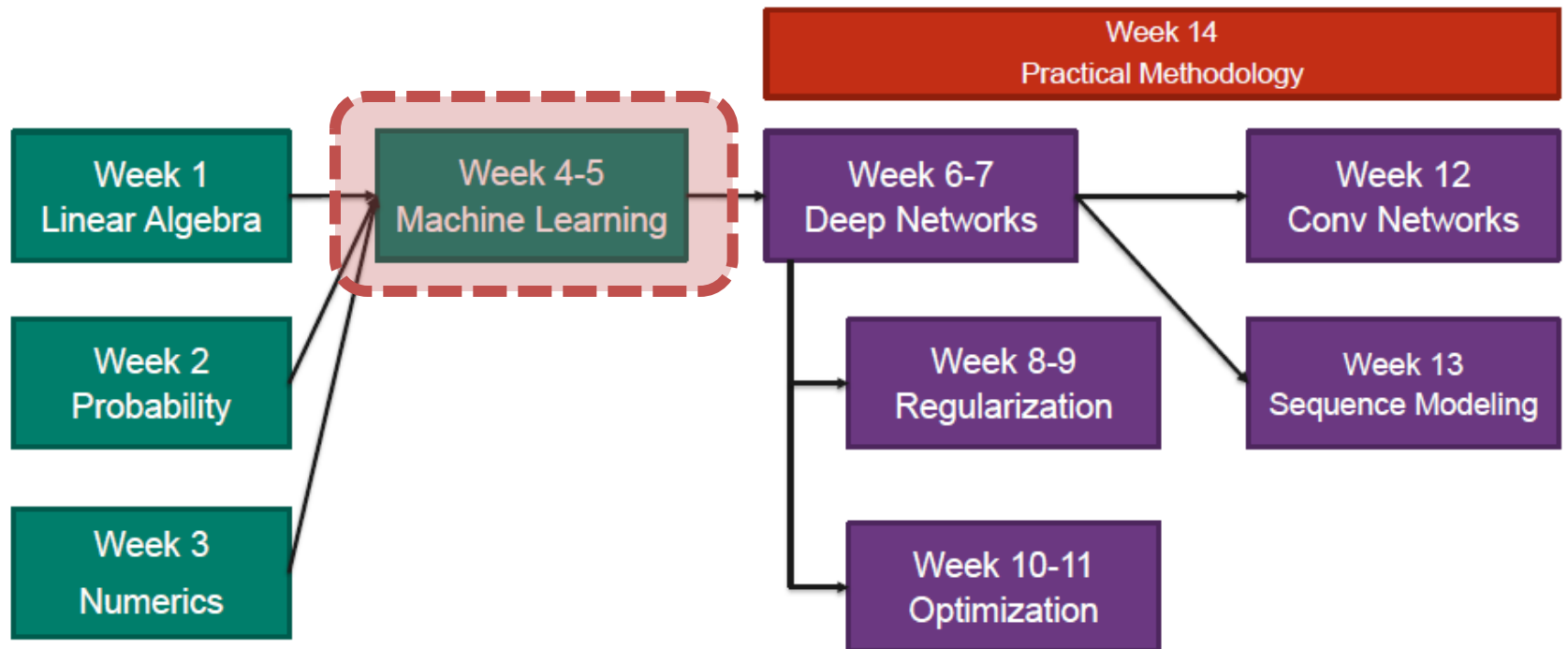


Hannes Badertscher

16 March 2026



Overview of DL-Lecture



Overview of this lecture

- Example: Linear Regression
- Capacity, Overfitting and Underfitting
- Estimators, Bias and Variance
- Maximum Likelihood Estimation
- Stochastic Gradient Descent
- Building a Machine Learning Algorithm
- Challenges Motivating Deep Learning

Week 4

Week 5

Recap from last week:

A machine learning algorithm improves a computers performance P at some task T via experience E .

- **Definition of machine learning algorithm:**

- **i.i.d. assumptions**

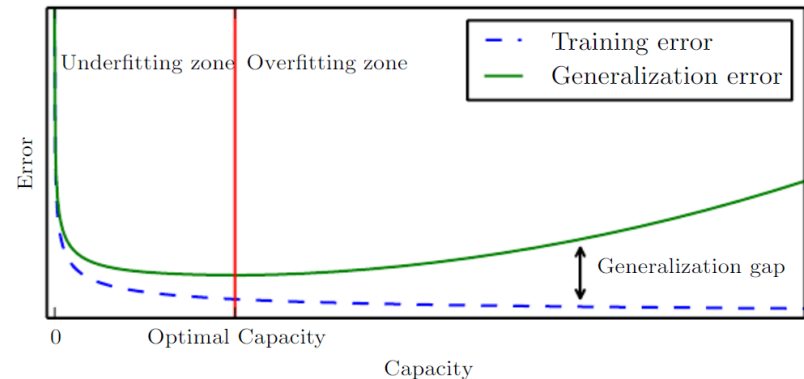
- Independent and identically distributed
- $\rightarrow$ allows to train on $\text{MSE}_{\text{train}}$ with the goal of reducing MSE_{test}

$$E[\text{MSE}_{\text{train}}] = E[\text{MSE}_{\text{test}}]$$

- **Capacity of a model**

- **Regularization**

Regularization is any modification we make to a learning algorithm that is intended to reduce its generalization error but not its training error



- **Bias**

$$\text{bias}(\hat{\theta}_m) = \mathbb{E}(\hat{\theta}_m) - \theta$$

- **Variance**

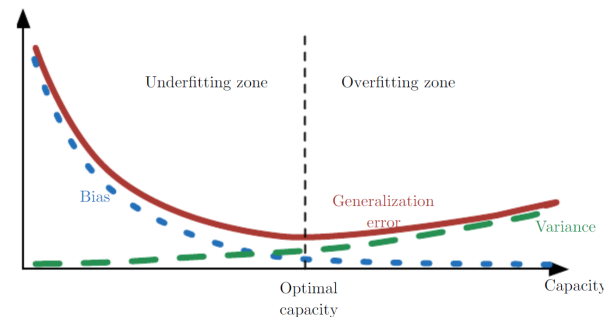
$$\text{Var}(\hat{\theta})$$

- **Consistency**

$$\lim_{m \rightarrow \infty} \hat{\theta}_m \xrightarrow{P} \theta$$

- **Bias-Variance Trade-Off**

$$\begin{aligned} \text{MSE} &= \mathbb{E}[(\hat{\theta}_m - \theta)^2] \\ &= \text{Bias}(\hat{\theta}_m)^2 + \text{Var}(\hat{\theta}_m) \end{aligned}$$



Overview of this lecture

- Example: Linear Regression
- Capacity, Overfitting and Underfitting
- Estimators, Bias and Variance
- Maximum Likelihood Estimation
- Stochastic Gradient Descent
- Building a Machine Learning Algorithm
- Challenges Motivating Deep Learning

Week 4

Week 5

Maximum Likelihood Estimation

- **Findings** from last week
 - Given an **estimator**, we can **calculate bias and variance** and judge the consistency
 - ➔ is there a way to find an **estimator** that is **guaranteed to be consistent**?
- **Yes ➔ Maximum Likelihood Estimation**
 - Guaranteed to be **consistent**
- **Goal: estimate true data-generating distribution** using data
 - ➔ estimate is not a value, but an entire probability distribution

$$p_{\text{data}}(\mathbf{x})$$

Maximum Likelihood Estimation

- **Approach**

- Collect dataset of i.i.d. samples
- Define parametric model distribution
- Find parameters θ of model by maximizing the likelihood

$$\mathbb{X} = \{\mathbf{x}^{(1)}, \dots, \mathbf{x}^{(m)}\}$$

$$p_{\text{model}}(\mathbf{x}; \theta)$$

$$\hat{\theta}_{\text{ML}} = \arg \max_{\theta} p_{\text{model}}(\mathbb{X}; \theta)$$

- **Core idea**

- All samples in the dataset have been observed → high likelihood

- This can be rewritten as

- Note: this relies on i.i.d.!

$$\hat{\theta}_{\text{ML}} = \arg \max_{\theta} \prod_{i=1}^m p_{\text{model}}(\mathbf{x}^{(i)}; \theta) \quad !$$

- → **Consistent estimator** for $p_{\text{data}}(x)$ for any parametric model

Maximum Likelihood Estimation

- **Problem:** Multiplying many small numbers is numerically not stable

- **Note:** the **logarithm** is a **monotonic** function, hence

$$\arg \max f(x) = \arg \max \log f(x)$$

- ➔ Maximize **log-likelihood**

$$= \arg \max_{\theta} \sum_{i=1}^m \log p_{\text{model}}(\mathbf{x}^{(i)}; \theta) \quad !$$

- This is equivalent to:

$$= \arg \max_{\theta} \left[\frac{1}{m} \sum_{i=1}^m \log p_{\text{model}}(\mathbf{x}^{(i)}; \theta) \right]$$

Relation to Cross-Entropy

- Given a dataset of i.i.d. data samples, the **empirical data distribution** is

$$\mathbb{X} = \{\mathbf{x}^{(1)}, \dots, \mathbf{x}^{(m)}\}$$

$$\hat{p}_{\text{data}}(\mathbf{x}) = \frac{1}{m} \sum_{i=1}^m \delta(\mathbf{x} - \mathbf{x}^{(i)})$$

- The **expectation over an empirical data distribution** is given by

$$\mathbb{E}_{\mathbf{x} \sim \hat{p}_{\text{data}}}[f(\mathbf{x})] = \frac{1}{m} \sum_{i=1}^m f(\mathbf{x}^{(i)})$$

- Hence, the term
can be rewritten as

$$\arg \max_{\theta} \frac{1}{m} \sum_{i=1}^m \log p_{\text{model}}(\mathbf{x}^{(i)}; \theta)$$

$$= \arg \max_{\theta} \mathbb{E}_{\mathbf{x} \sim \hat{p}_{\text{data}}} \log p_{\text{model}}(\mathbf{x}; \theta) \quad !$$

Relation to Cross-Entropy

- Recap from week 2: **cross-entropy** is given as:

$$H(P, Q) = -\mathbb{E}_{x \sim P} \log Q(x)$$

- Maximizing the log-likelihood**
=
Minimizing the cross-entropy
=
Minimizing the Kullback-Leibler divergence

$$\hat{\theta}_{ML} = \arg \max_{\theta} \mathbb{E}_{\mathbf{x} \sim \hat{p}_{data}} \log p_{model}(\mathbf{x}; \theta)$$

$$\hat{\theta}_{ML} = \arg \min_{\theta} H(\hat{p}_{data}, p_{model})$$

!

$$\hat{\theta}_{ML} = \arg \min_{\theta} D_{KL}(\hat{p}_{data} || p_{model})$$

→ **Standard method** for training DL models:
minimize cross-entropy

Estimating conditional probabilities

- So far: estimate a generic probability density
- For **supervised machine learning**, a **conditional probability** is estimated instead
 - $\mathbf{x}$: measured input data
 - $\mathbf{y}$: output
- **Maximum (log-) likelihood or minimizing the cross-entropy** works exactly the same way

$$p_{model}(\mathbf{x}; \boldsymbol{\theta})$$

$$p_{model}(\mathbf{y} | \mathbf{x}; \boldsymbol{\theta})$$

$$\hat{\theta}_{\text{ML}} = \arg \max_{\theta} \sum_{i=1}^m \log P(\mathbf{y}^{(i)} | \mathbf{x}^{(i)}; \theta) \quad !$$

Example: Linear Regression

- Goal of linear regression so far:
minimize the MSE

- **MSE: arbitrary selection**, why not something else?

$$\hat{y} = \mathbf{w}^\top \mathbf{x}$$

$$\hat{\mathbf{w}} = \arg \min_{\mathbf{w}} \text{MSE}_{\text{train}}$$

- New approach: train **linear regression** using **cross-entropy**

- ➔ What is the **parametric model distribution**?

$$p_{\text{model}}(y | \mathbf{x}; \boldsymbol{\theta})$$

- **Assumptions**

- On average, a **linear model** is correct
- **Random error** that follows a **Gaussian** distribution

- ➔ Resulting **model distribution**

- Note: σ^2 is unknown, will be dealt with later

$$p_{\text{model}}(y | \mathbf{x}; \mathbf{w}) = \mathcal{N}(y; \underbrace{\hat{y}(\mathbf{x}; \mathbf{w})}_{\text{mean } \mu}, \sigma^2)$$

mean μ

!

Example: Linear Regression

- Given p_{model} , the **optimal parameters** can be found using maximum likelihood
- Without proof, plugging the **Gaussian distribution** into **maximum likelihood** results in:
 - **Blue**: constant terms $\rightarrow$ ignore
 - **Green**: constant factor $\rightarrow$ ignore
 - $\rightarrow$ independent of σ^2
- This results in:
 - Because $1/m$ is just a constant factor
- Minimizing $\text{MSE}_{\text{train}}$** is equivalent to performing **maximum likelihood** with a **Gaussian assumption**

$$\hat{\theta}_{\text{ML}} = \arg \max_{\theta} \sum_{i=1}^m \log P(\mathbf{y}^{(i)} \mid \mathbf{x}^{(i)}; \theta)$$

$$\boxed{-m \log \sigma - \frac{m}{2} \log(2\pi)} - \sum_{i=1}^m \frac{\|\hat{y}^{(i)} - y^{(i)}\|^2}{\boxed{2\sigma^2}}$$

$$\hat{\theta}_{\text{ML}} = \arg \max_{\theta} - \sum_{i=1}^m \|\hat{y}^{(i)} - y^{(i)}\|^2$$

$$\hat{\theta}_{\text{ML}} = \arg \min_{\theta} \boxed{\frac{1}{m} \sum_{i=1}^m \|\hat{y}^{(i)} - y^{(i)}\|^2}$$

Summary

- **Maximum Likelihood**
 - **Consistent estimator** = highly desirable
- Equivalent to **minimizing the cross-entropy** between the **empirical data distribution** and the **parametric model distribution**
- **Supervised learning** → conditional probability
- **Linear regression**: assumption of Gaussian errors with maximum likelihood = minimize $\text{MSE}_{\text{train}}$

$$\hat{\theta}_{\text{ML}} = \arg \max_{\theta} \mathbb{E}_{\mathbf{x} \sim \hat{p}_{\text{data}}} \log p_{\text{model}}(\mathbf{x}; \theta)$$

$$H(\hat{p}_{\text{data}}, p_{\text{model}}) = -\mathbb{E}_{\mathbf{x} \sim \hat{p}_{\text{data}}} [\log p_{\text{model}}]$$

$$\hat{\theta}_{\text{ML}} = \arg \max_{\theta} \sum_{i=1}^m \log P(\mathbf{y}^{(i)} \mid \mathbf{x}^{(i)}; \theta)$$

$$p(y \mid \mathbf{x}) = \mathcal{N}(y; \hat{y}(\mathbf{x}; \mathbf{w}), \sigma^2)$$

$$\text{MSE}_{\text{train}} = \frac{1}{m} \sum_{i=1}^m \|\hat{y}^{(i)} - y^{(i)}\|^2$$

Overview of this lecture

- Example: Linear Regression
- Capacity, Overfitting and Underfitting
- Estimators, Bias and Variance

Week 4

-
- Maximum Likelihood Estimation

Week 5

- Stochastic Gradient Descent

- Building a Machine Learning Algorithm
- Challenges Motivating Deep Learning

Stochastic Gradient Descent

- So far: train machine learning algorithm using **maximum likelihood** method
- Two weeks ago: gradient descent to minimize a function
- Maximum likelihood → sum over all m training examples
- **Gradient** → also a sum over all m training examples
- → **huge computational effort** for one **tiny step**
- → **not feasible**

$$\hat{\theta}_{\text{ML}} = \arg \max_{\theta} \sum_{i=1}^m \log P(\mathbf{y}^{(i)} | \mathbf{x}^{(i)}; \theta)$$

$$\theta' = \theta - \epsilon \nabla_{\theta} f(\theta)$$

Stochastic Gradient Descent

- Maximum likelihood can be rewritten as follows:
 - Maximizing is equivalent to minimizing the inverse
 - Constant factors don't change θ

$$\begin{aligned}\hat{\theta}_{\text{ML}} &= \arg \max_{\theta} \sum_{i=1}^m \log P(y^{(i)} | x^{(i)}; \theta) \\ &= \arg \min_{\theta} \frac{1}{m} \sum_{i=1}^m -\log P(y^{(i)} | x^{(i)}; \theta)\end{aligned}$$

- This is equivalent to:
 - $J(\theta)$ is the **loss function** to minimize
 - $L(x, y, \theta)$ is the **per-sample loss**

- ➔ The **loss function** is the **average of a per-sample loss**

- ➔ **Expectation over an empirical distribution**

$$\hat{\theta}_{\text{ML}} = \arg \min_{\theta} J(\theta)$$

where $J(\theta) = \frac{1}{m} \sum_{i=1}^m L(x^{(i)}, y^{(i)}, \theta)$!

and $L(x, y, \theta) = -\log p(y | x; \theta)$

$$J(\theta) = \mathbb{E}_{\mathbf{x}, y \sim \hat{p}_{\text{data}}} L(\mathbf{x}, y, \theta) \quad !$$

Stochastic Gradient Descent

- The **gradient of the loss function** can be calculated as

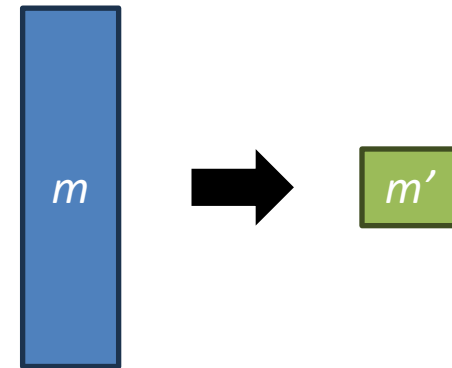
- ➔ each training step involves **averaging** the **per-sample gradient** over all m training examples
- ➔ also an **expectation** over the **empirical distribution**

$$\nabla_{\theta} J(\theta) = \frac{1}{m} \sum_{i=1}^m \nabla_{\theta} L(x^{(i)}, y^{(i)}, \theta)$$

$$\nabla_{\theta} J(\theta) = \mathbb{E}_{x, y \sim \hat{p}_{data}} [\nabla_{\theta} L(x, y, \theta)]$$

- **Key idea** of SGD

- **Approximate the gradient with fewer samples**
- $\nabla_{\theta} J(\theta)$ is the true gradient
- $\hat{g}$ is an approximation using the sample mean as an estimator
- ➔ unbiased, but some variance



$$\hat{g} = \frac{1}{m'} \sum_{i=1}^{m'} \nabla_{\theta} L(x^{(i)}, y^{(i)}, \theta) \quad !$$

Stochastic Gradient Descent

- **Minibatch $\mathbb{B}$:** random subset of training set

$$\mathbb{B} = \{\mathbf{x}^{(1)}, \dots, \mathbf{x}^{(m')}\}$$

- Minibatch size $m' \rightarrow$ important hyperparameter
- Typically, between 1 and 1'000
- Independent from dataset size!
- Often selected based on available GPU memory

$$\mathbf{g} = \frac{1}{m'} \sum_{i=1}^{m'} \nabla_{\boldsymbol{\theta}} L(\mathbf{x}^{(i)}, y^{(i)}, \boldsymbol{\theta})$$

- **SGD Algorithm**

1. Randomly sample minibatch $\mathbb{B}$ of size m'
2. Estimate gradient using minibatch

$$\hat{\mathbf{g}} = \frac{1}{m'} \sum_{i=1}^{m'} \nabla_{\boldsymbol{\theta}} L(\mathbf{x}^{(i)}, y^{(i)}; \boldsymbol{\theta})$$

3. Take small step in negative gradient direction
$$\boldsymbol{\theta} \leftarrow \boldsymbol{\theta} - \epsilon \hat{\mathbf{g}}$$

Overview of this lecture

- Example: Linear Regression
- Capacity, Overfitting and Underfitting
- Estimators, Bias and Variance
- Maximum Likelihood Estimation
- Stochastic Gradient Descent
- Building a Machine Learning Algorithm
- Challenges Motivating Deep Learning

Week 4

Week 5

Building a Machine Learning Algorithm

- **Required ingredients** to build a machine learning algorithm:
- Each can be changed **independently**

- A (large) data set
- A cost/loss function
- A probabilistic model
- An optimization algorithm

- **Example:** Linear Regression
 - **Dataset:** measured data
 - **Cost-function:** cross-entropy
 - **Probabilistic model:** Gaussian
 - **Optimization algorithm:** normal equations

$$y \in \mathbb{R} \quad \mathbf{x} \in \mathbb{R}^n$$

$$J(\mathbf{w}, b) = -\mathbb{E}_{\mathbf{x}, y \sim \hat{p}_{\text{data}}} \log p_{\text{model}}(y \mid \mathbf{x})$$

$$p_{\text{model}}(y \mid \mathbf{x}) = \mathcal{N}(y; \mathbf{x}^\top \mathbf{w} + b, 1)$$

$$\mathbf{w} = \left(\mathbf{X}^{(\text{train})\top} \mathbf{X}^{(\text{train})} \right)^{-1} \mathbf{X}^{(\text{train})\top} \mathbf{y}^{(\text{train})}$$

Building a Machine Learning Algorithm

- **Default cost function** → cross-entropy
 - **Regularization** → can add certain terms to the cost function
- **Default probabilistic model** → depends on output data
 - More details: next week
- **Default optimization algorithm**
 - Usually: no closed-form solution
 - → Stochastic gradient descent (**SGD**)

$$J(\mathbf{w}, b) = -\mathbb{E}_{\mathbf{x}, y \sim \hat{p}_{\text{data}}} \log p_{\text{model}}(y | \mathbf{x})$$

$$J(\mathbf{w}, b) = \boxed{\lambda \|\mathbf{w}\|_2^2} - \mathbb{E}_{\mathbf{x}, y \sim p_{\text{data}}} \log p_{\text{model}}(y | \mathbf{x})$$

Overview of this lecture

- Example: Linear Regression
- Capacity, Overfitting and Underfitting
- Estimators, Bias and Variance
- Maximum Likelihood Estimation
- Stochastic Gradient Descent
- Building a Machine Learning Algorithm
- Challenges Motivating Deep Learning

Week 4

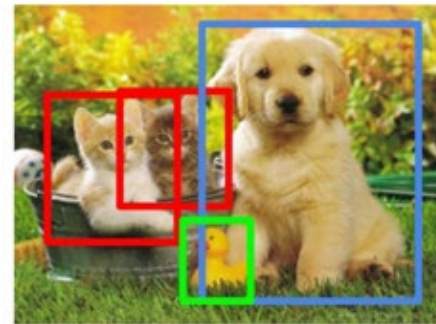
Week 5

Challenges Motivating Deep Learning

Why do we need deep learning?

- **Traditional algorithms** and **machine learning** work well in many applications
- But: many **central problems** were **not solved**
 - Recognizing speech
 - Understanding / processing text
 - Detecting objects on images
- ➔ **Why** is this?

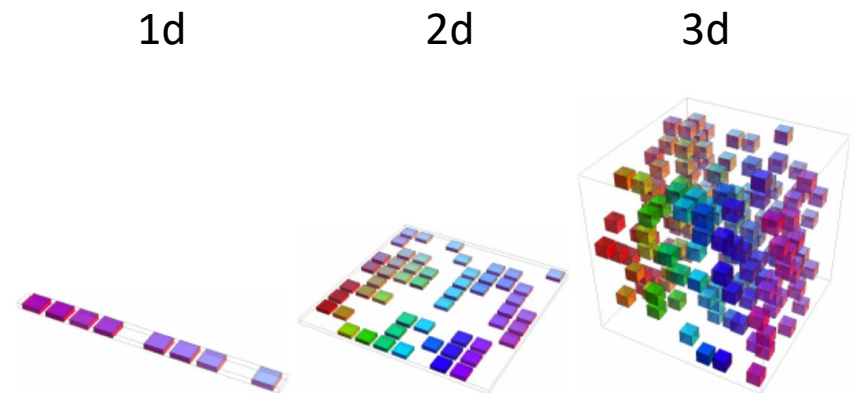
Object Detection



CAT, DOG, DUCK

The Curse of Dimensionality

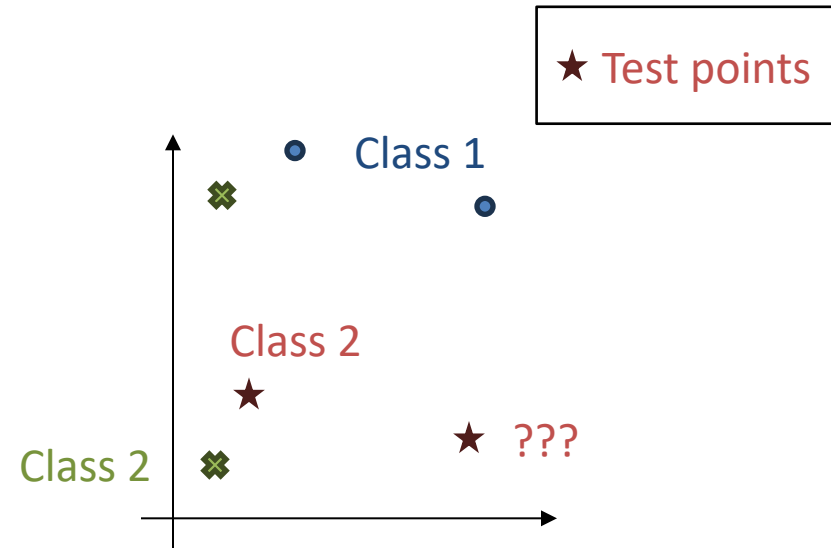
- As the **number of variables increase**, the **number of combinations** increases exponentially
- Example:
 - 10 possible values per dimension
 - Target: know 80% of possible values
 - **1d** → requires **8** values
 - **2d** → requires **64** values
 - **3d** → requires **512** values
- **Number of training samples** → must **increase exponentially**
 - → Data collection, memory consumption, processing power also increases exponentially



DL, Fig. 5.9

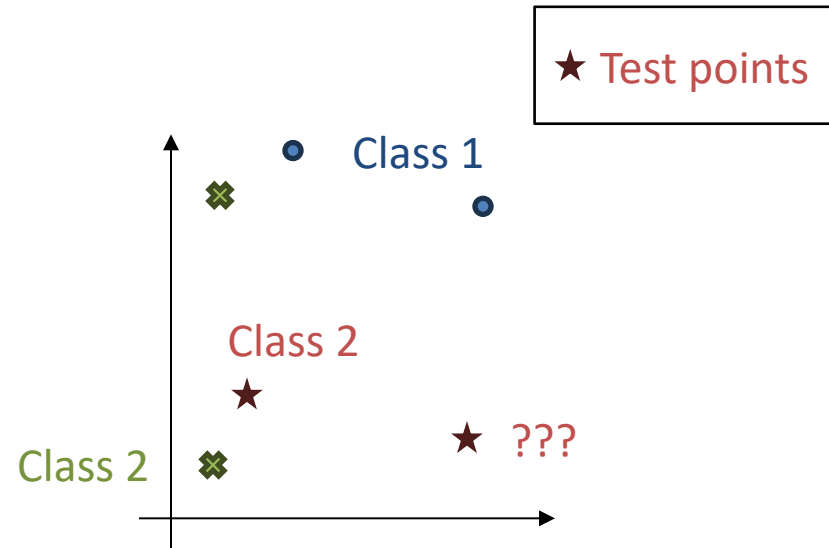
The Curse of Dimensionality

- Collecting such amounts of data is not feasible
- → for high-dimensional data **most of the space is empty**
- → What to predict for test points that are far from all training points?



Traditional ML

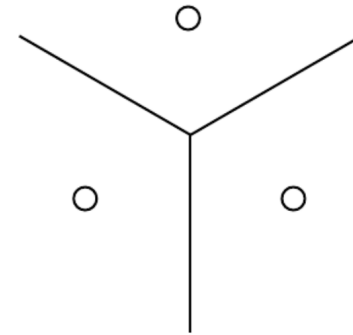
- Generalization is based on assumptions about the data
- Traditional ML → assumption that **data is locally constant or smooth!**
- Mathematically, this can be written as shown on the right
- In **high-dimensional data**, this **assumption** fails
 - In mostly empty space, **all points are far away**



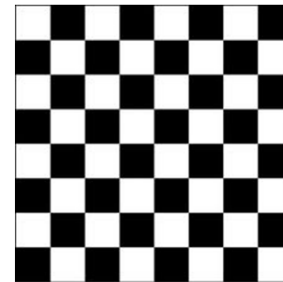
$$f^*(x) \approx f^*(x + \epsilon)$$

Traditional ML

- **Classical ML** that assumes only **smoothness**:
 - For m **regions**, at least m **training points** are required
 - Here: 3 regions → 3 examples
- For **complex patterns**, this is not efficient
 - 8x8 checkerboard → 64 regions → 64 examples
- More **abstract assumptions** – e.g. assuming the pattern repeats – would require much **less data**



DL, Fig. 5.10



Additional assumptions

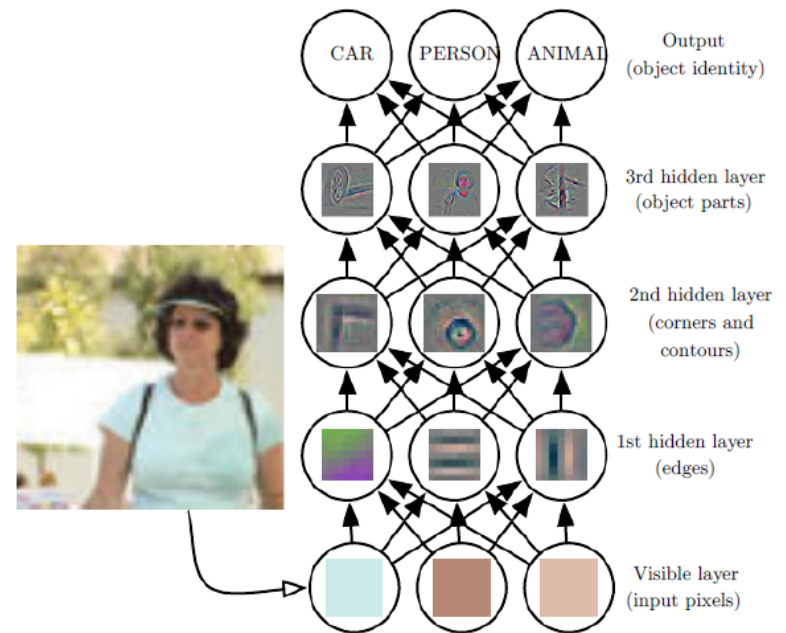
- How to **generalize from few observations** → **additional assumptions**
- **Example:**
 - Default: 9 regions → 9 examples
 - Possible **assumption**: X_1 and X_2 are **statistically independent**
 - → **3 examples** for X_1
 - → **3 examples** for X_2
 - → **only 6 instead of 9 examples**
- But: what **assumption** for more **complex patterns**?

	$X_1 = 0$	$X_1 = 1$	$X_1 = 2$
$X_2 = 0$			
$X_2 = 1$			
$X_2 = 2$			

$$p(X_1, X_2) = p(X_1)p(X_2)$$

Assumption in Deep Learning

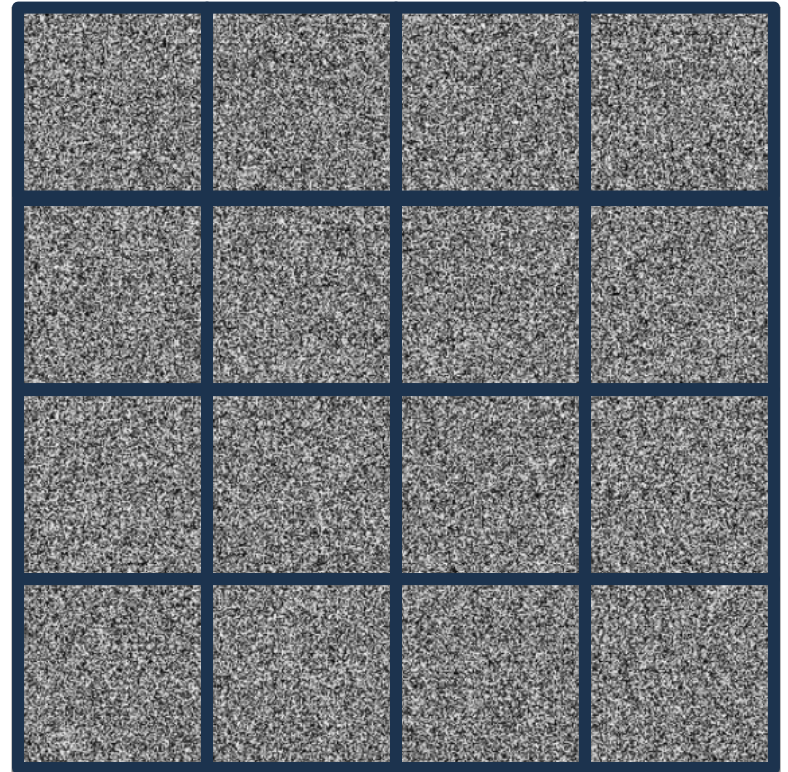
- **Assumption:** output was generated by a **composition of factors** over multiple layers
 - ➔ **Output** depends on high-level features
 - ➔ **High-level features** depend on mid-level features
 - ➔ **Mid-level features** depend on low-level features
 - ➔ **Low-level features** depend on input values
- Simple, **mild assumption**, but **exponential relationship** between input values and output
 - ➔ **Counters curse of dimensionality**



DL, Fig. 1.2

Manifolds

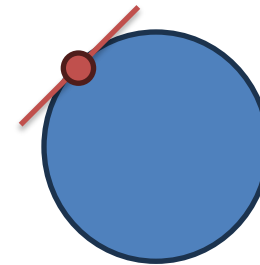
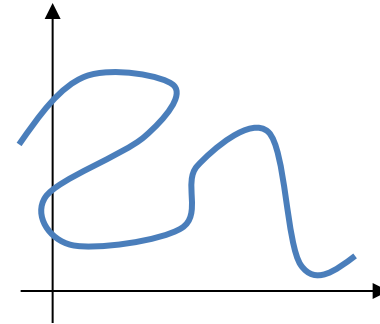
- **Curse of dimensionality** → most of the **space is empty**
- But: much of the **space CAN be empty** because it is **irrelevant**
- **Example:**
 - Right: 16 images that are **randomly sampled** from the **space of all possible images**
 - → Such images are **not relevant for classifying objects**
- Only the **space of “relevant” data** needs to be analyzed



DL, Fig. 5.12

Manifolds

- **Manifold: connected region** or set of points that is **embedded in a high-dimensional space**, but only **spans a smaller number of dimensions**
- **Example:**
 - Our **world** is a **3-D space**
 - **Locally** at any point, it can be approximated as a **2-D manifold**
 - → we don't always need to use 3-D coordinates
 - A **street** is a **1-D manifold** in 3-D space → only one coordinate required

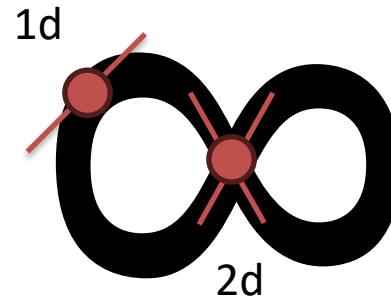


Manifolds

- **Dimensionalities of manifolds** can **vary**, e.g. at self-intersections

- **Example**

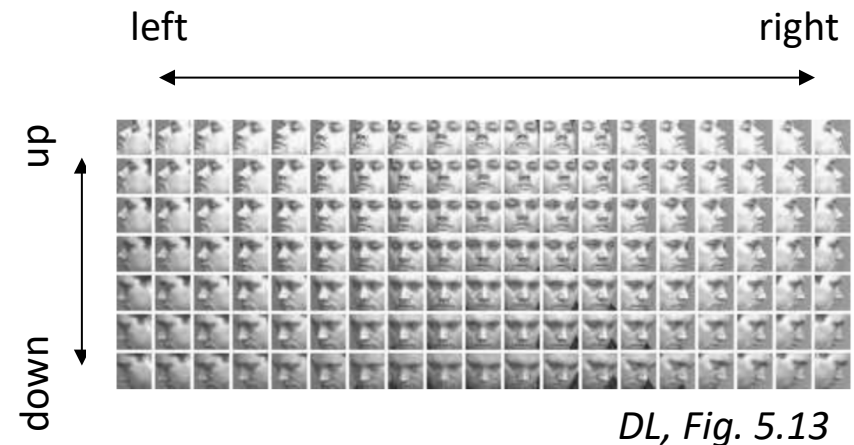
- Intersection: 2-d manifold
- Other points: 1-d manifold



- **Assumption** to counter curse of dimensionality: **all points lie on a manifold**
 - → **Training data** is not required in an exponentially large space → only on a **smaller-dimensional manifold**

Manifolds

- **Natural images** lie on a low-dimensional manifold
 - Only **few combinations of pixel values** actually make sense
 - These are connected through (mathematical) transformations
- Often, data will lie on **multiple manifolds**
 - Example: a **manifold of cats** and a separate **manifold of dogs**
- Directly **extracting manifolds** and meaningful **coordinates** (like in the example) is **difficult**
- ➔ and usually **not required**



DL, Fig. 5.13

Overview of this lecture

- Example: Linear Regression
- Capacity, Overfitting and Underfitting
- Estimators, Bias and Variance
- Maximum Likelihood Estimation
- Stochastic Gradient Descent
- Building a Machine Learning Algorithm
- Challenges Motivating Deep Learning

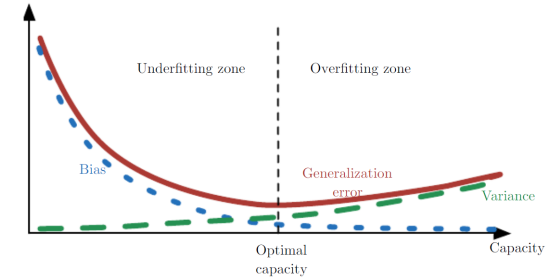
Week 4

Week 5

Summary

- **Bias Variance Trade-Off**

$$\text{MSE} = \text{Bias}(\hat{\theta}_m)^2 + \text{Var}(\hat{\theta}_m)$$



- **Maximum Likelihood**

- The Maximum Likelihood method is given by
- Which is equivalent to **minimizing the cross-entropy** between the empirical data distribution and the model distribution
- It can be shown that the maximum likelihood is a **consistent estimator**!
- For conditional likelihoods, the maximum likelihood is given by
- In the **linear regression** case, assuming the errors are Gaussian, minimizing the cross-entropy is equivalent to minimizing the MSE

$$\hat{\theta}_{\text{ML}} = \arg \max_{\theta} p_{\text{model}}(\mathbb{X}; \theta)$$

$$= \arg \max_{\theta} \sum_{i=1}^m \log p_{\text{model}}(\mathbf{x}^{(i)}; \theta)$$

$$H(\hat{p}_{\text{data}}, p_{\text{model}}) = -\mathbb{E}_{\mathbf{x} \sim \hat{p}_{\text{data}}} [\log p_{\text{model}}]$$

$$\hat{\theta}_{\text{ML}} = \arg \max_{\theta} \sum_{i=1}^m \log P(\mathbf{y}^{(i)} \mid \mathbf{x}^{(i)}; \theta)$$

$$p(y \mid \mathbf{x}) = \mathcal{N}(y; \hat{y}(\mathbf{x}; \mathbf{w}), \sigma^2)$$

$$\text{MSE}_{\text{train}} = \frac{1}{m} \sum_{i=1}^m \|\hat{y}^{(i)} - y^{(i)}\|^2$$

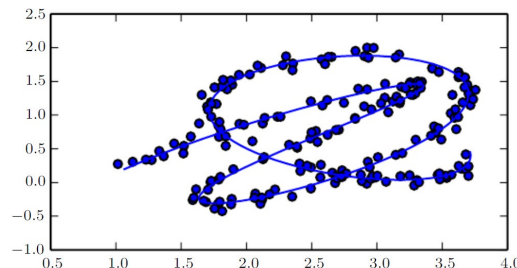
Summary:

- **Stochastic Gradient Descent (SGD)**
 - In gradient descent, a sum over all m training examples is used to estimate the gradient.
 - In SGD, the gradient is estimated over a small mini-batch of size $m' \ll m$
- **Building a Machine Learning Algorithm**
- **Challenges Motivating Deep Learning**
 - Failure of other methods
 - Curse of Dimensionality
 - Assumptions:
 - Local consistency
 - Composition of factors
 - Manifolds

$$\nabla_{\theta} J(\theta) = \frac{1}{m} \sum_{i=1}^m \nabla_{\theta} L(\mathbf{x}^{(i)}, y^{(i)}, \theta)$$
$$\mathbf{g} = \frac{1}{m'} \sum_{i=1}^{m'} \nabla_{\theta} L(\mathbf{x}^{(i)}, y^{(i)}, \theta)$$

- A (large) data set
- A cost/loss/objective function
- A probabilistic model
- An optimization algorithm

$$f^*(\mathbf{x}) \approx f^*(\mathbf{x} + \epsilon)$$



Data was generated hierarchically over several levels by the composition of factors

Overview of DL-Lecture

